

Distributed Memory Programming with MPI

Pacheco chapter 3

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Copyright © 2013, 2014, 2017–2026
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/CP/>



Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Credits

- Peter Pacheco, Dept. of Computer Science, University of San Francisco
<http://www.cs.usfca.edu/~peter/>
- Mary Hall, School of Computing, University of Utah
<https://www.cs.utah.edu/~mhall/>
- Salvatore Orlando, Univ. Ca' Foscari di Venezia
<http://www.dsi.unive.it/~orlando/>
- Blaise Barney,
<https://hpc-tutorials.llnl.gov/mpi/> ([highly recommended!!](#))

Introduction

Message Passing and MPI

- **Message passing** is the predominant programming model for supercomputers and clusters.
- What MPI is:
 - A library used within conventional sequential languages (Fortran, C, C++).
 - Based on Single Program, Multiple Data (SPMD) paradigm.
 - Isolation of separate address spaces.
 - no data races, but communication errors possible.
 - exposes execution model and forces programmer to think about locality, both good for performance.
 - complexity and code growth!

SPMD

(Single Program Multiple Data)

- The **same** program is executed by P processes.
- Each process may choose a different execution path depending on its ID (*rank*).

```
...  
MPI_Init(...);  
...  
foo();                               /* executed by all processes */  
if ( my_rank == 0 ) {  
    do_something();                   /* executed by process 0 only */  
} else {  
    do_something_else();              /* executed by all other processes */  
}  
...  
MPI_Finalize();  
...
```

Message Passing and MPI

- All communication and synchronization operations require subroutine calls.
 - No shared variables.
- Subroutines for:
 - **Communication**
 - Pairwise or point-to-point.
 - Collectives involving multiple processes.
 - **Synchronization**
 - Barrier.
 - No locks because there are no shared variables to protect.
 - **Queries**
 - How many processes? Which one am I? Any messages waiting?

Using MPI under Debian/Ubuntu

- Install the `mpi-default-bin` and `mpi-default-dev` packages
 - Installs Open MPI.
 - You might also want `openmpi-doc` for the man pages.
- Use `mpicc` to compile, `mpirun` to execute.
- To execute your program on remote hosts, make sure you can ssh into them without entering a password.
 - If you have not already done so, generate a public/private key pair on your local machine with `ssh-keygen`; do not enter a passphrase.
 - Copy the public key to the remote machine:

```
ssh-copy-id remote-user@remote-host
```


Finding Out About the Environment

- Two important questions that arise frequently in a parallel program are:
 - How many processes are taking part in this computation?
 - Who am I?
- MPI provides functions to answer these questions.
 - `MPI_Comm_size()` reports the number of processes.
 - `MPI_Comm_rank()` reports the *rank*, a number between 0 and (*size* - 1), identifying the calling process.

Hello, world!

```
/* mpi-hello.c */

#include <stdio.h>
#include <mpi.h>

int main( int argc, char *argv[] )
{
    int rank, size, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    MPI_Get_processor_name( hostname, &len );
    printf("Greetings from process %d of %d running on %s\n",
           rank, size, hostname);
    MPI_Finalize();
    return 0;
}
```

No MPI call before this line

No MPI call after this line

Hello, world!

- Compilation:

```
mpicc -Wall mpi-hello.c -o mpi-hello
```

- Execution (8 processes on localhost):

```
mpirun -n 8 ./mpi-hello
```

- Execution (two processes on host “foo” and one on host “bar”)

```
mpirun -H foo,foo,bar ./mpi-hello
```

Hello, world!

```
$ mpirun -n 8 ./mpi-hello
```

```
Greetings from process 7 of 8 running on wopr
```

```
Greetings from process 5 of 8 running on wopr
```

```
Greetings from process 0 of 8 running on wopr
```

```
Greetings from process 3 of 8 running on wopr
```

```
Greetings from process 6 of 8 running on wopr
```

```
Greetings from process 4 of 8 running on wopr
```

```
Greetings from process 1 of 8 running on wopr
```

```
Greetings from process 2 of 8 running on wopr
```

Running on the HPC cluster

- Prepare a script, e.g., **run-mpi-program.sh**

```
#!/bin/bash

#SBATCH --time=00:05:00
#SBATCH -n 4
#SBATCH --output slurm-%j.out
#SBATCH --partition=rtx2080

echo "== Compiling =="
mpicc -std=c99 -Wall -Wpedantic mpi-program.c -o mpi-program

echo "== Running MPI program with $SLURM_NTASKS cores =="
srun -mpi=pmix ./mpi-program

echo "== End of Job =="
```

- **sbatch -n *n_of_cores* run-mpi-program.sh**
 - Note: the maximum number of cores allowed on the cluster is **8**.

MPI

- The following six functions suffice for simple programs:
 - `MPI_Init`
 - `MPI_Finalize`
 - `MPI_Comm_size` (how many processes are there?)
 - `MPI_Comm_rank` (who am I?)
 - `MPI_Send` (blocking send)
 - `MPI_Recv` (blocking receive)
 - `MPI_Abort` (aborting the computation)

A Simple MPI Program

```
/* mpi-point-to-point.c */
#include <stdio.h>
#include <mpi.h>
int main( int argc, char *argv[])
{
    int my_rank, buf;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    /* process 0 sends and process 1 receives */
    if (my_rank == 0) {
        buf = 123456;
        MPI_Send(&buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (my_rank == 1) {
        MPI_Recv(&buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        printf("Received %d\n", buf);
    }

    MPI_Finalize();
    return 0;
}
```

Some Basic Concepts

- How processes are organized:
 - Processes can be collected into **groups**.
 - A group and context together form a **communicator**.
 - A process is identified by its rank in the group associated with a communicator.
- There is a default communicator **MPI_COMM_WORLD** whose group contains all processes.

MPI datatypes

- Data sent or received is described by a triple (**address**, **count**, **datatype**).
- An MPI datatype is recursively defined as:
 - a predefined type (e.g., **MPI_INT**, **MPI_DOUBLE**).
 - a contiguous array of MPI datatypes.
 - a *strided* block of MPI datatypes.
 - an indexed array of blocks of MPI datatypes.
 - an arbitrary structure of MPI datatypes.
- There are MPI functions to define custom datatypes.

Some MPI default datatypes

MPI Datatype	C datatype
<code>MPI_CHAR</code>	signed char
<code>MPI_SHORT</code>	signed short int
<code>MPI_INT</code>	signed int
<code>MPI_LONG</code>	signed long int
<code>MPI_LONG_LONG</code>	signed long long int
<code>MPI_UNSIGNED_CHAR</code>	unsigned char
<code>MPI_UNSIGNED_SHORT</code>	unsigned short int
<code>MPI_UNSIGNED</code>	unsigned int
<code>MPI_UNSIGNED_LONG</code>	unsigned long int
<code>MPI_FLOAT</code>	float
<code>MPI_DOUBLE</code>	double
<code>MPI_LONG_DOUBLE</code>	long double
<code>MPI_BYTE</code>	

MPI Tags

- Messages are sent with an accompanying user-defined integer *tag*, to assist the receiving process in identifying the message.
- Messages can be screened at the receiving end by specifying a tag, or not screened by specifying **MPI_ANY_TAG** as the tag.
 - If the receiver gets a message with a different tag than the one specified in the **MPI_Recv()** call, the message is kept on hold and will be matched by a future **MPI_Recv()** with the correct tag.

Blocking Send

```
int buf = 123456;  
MPI_Send(&buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

`int MPI_Send(const void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)`

- The message buffer is described by (buf, count, datatype).
- count is the number of *items* to send (NOT the number of *bytes*)
- dest is the rank of the target process
 - If dest is `MPI_PROC_NULL`, the `MPI_Send` operation has no effect
- tag is the message tag (we will always use 0)
- When this function returns, the data has been delivered to the system and the buffer can be reused. The message may have not been received yet by the target process.

Blocking Receive

```
MPI_Recv( &buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status );
```

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,  
int source, int tag, MPI_Comm comm, MPI_Status *status)
```

- Waits until a matching (both **source** and **tag**) message is received, and the buffer can be used
- **source** is the process rank in the communicator specified by comm, or **MPI_ANY_SOURCE**
 - If source is **MPI_PROC_NULL**, the **MPI_Recv** operation has no effect
- **tag** is a tag to be matched, or **MPI_ANY_TAG**
- receiving fewer than **count** items is OK, but receiving more is an error (see later)
- **status** contains further information (e.g., message size), use **MPI_STATUS_IGNORE** if no information is required

Send/Receive message sizes

Examples

- Sender sends 10, receiver expects (at most) 10 → OK
- Sender sends 10, receiver expects (at most) 20 → OK
- Sender sends 20, receiver expects (at most) 10 → **MPI_ERR_TRUNCATE**
- **MPI_Recv()** ensures that the receiver always gets the full message that was sent
 - If 10 items are sent, a matching **MPI_Recv()** will return when all 10 items have been received

MPI_Status

- **MPI_Status** is a C structure with (among others) the following fields:
 - `int MPI_SOURCE;`
 - `int MPI_TAG;`
 - `int MPI_ERROR;`
- Therefore, a process can check the actual source and tag of a message received with **MPI_ANY_TAG** or **MPI_ANY_SOURCE**

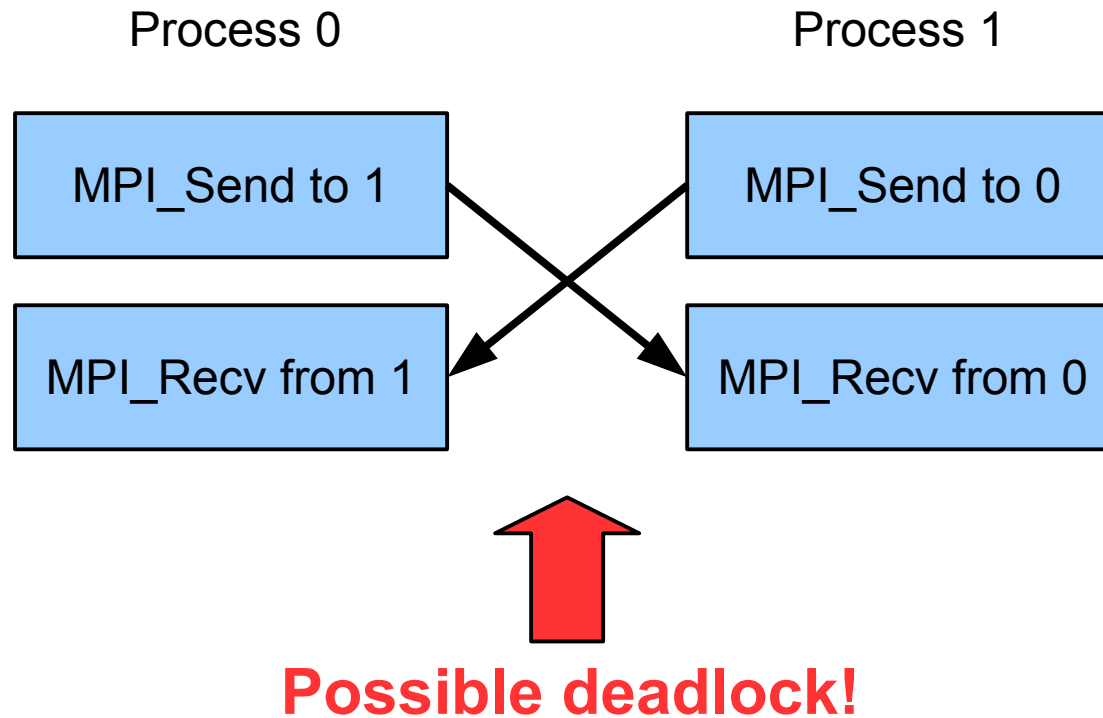
MPI_Get_count()

```
int MPI_Get_count( const MPI_Status *status,  
MPI_Datatype datatype, int *count )
```

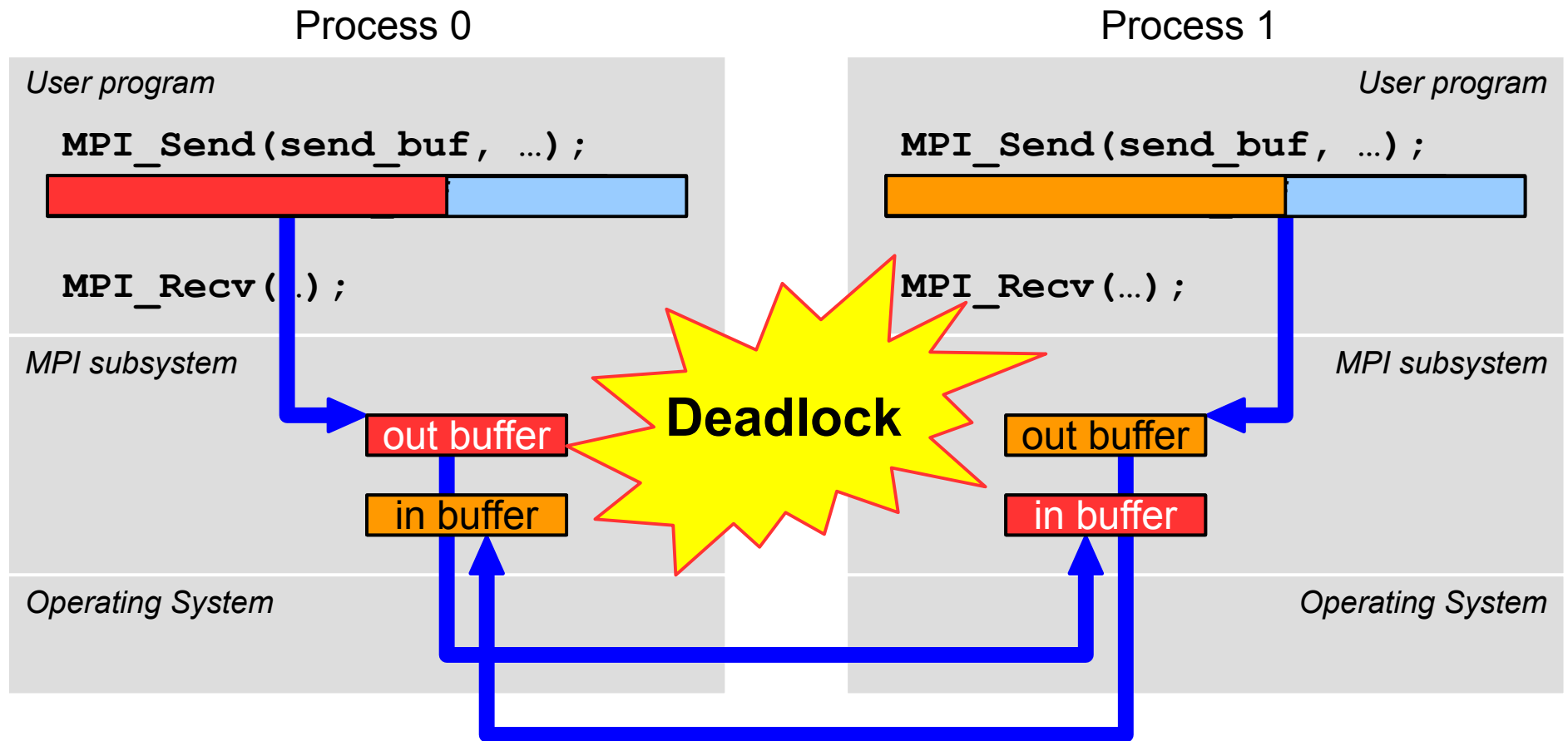
- **MPI_Recv** may complete even if less than *count* elements have been received
 - Provided that the matching **MPI_Send** actually sent fewer elements
- **MPI_Get_count** can be used to know how many elements of type *datatype* have actually been received
- See [mpi-get-count.c](#)

Blocking communication and deadlocks

- Blocking send/receive **may** lead to deadlock if not paired carefully

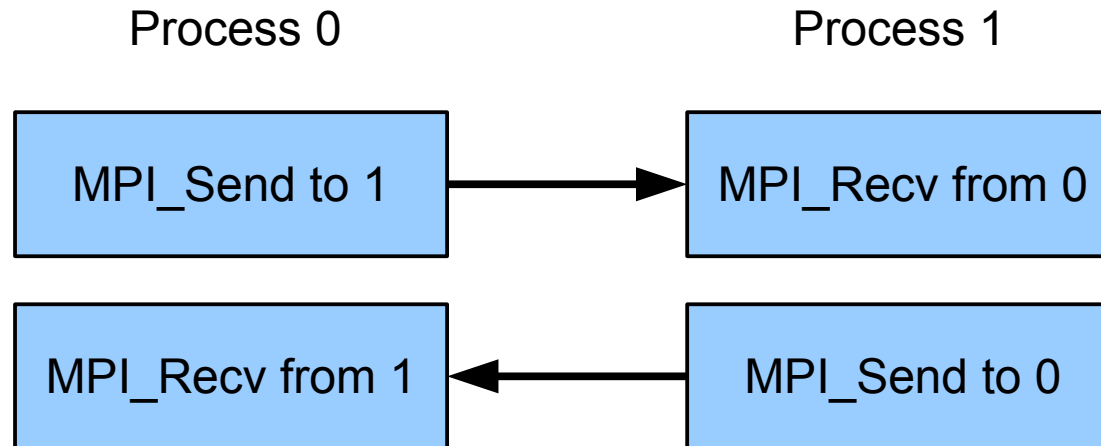


Blocking communication and deadlocks



Blocking communication and deadlocks

- To avoid the deadlock it is possible to reorder the operations so that send/receive pairs match...



- ...or use **non-blocking** communication primitives
- ...or use **MPI_Sendrecv()** if appropriate (see later)

Non-blocking Send

```
int buf = 123456;  
MPI_Request req;  
MPI_Isend(&buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &req);
```

```
int MPI_Isend(const void *start, int count,  
MPI_Datatype datatype, int dest, int tag,  
MPI_Comm comm, MPI_Request *req)
```

- The message buffer is described by (start, count, datatype).
- **count** is the number of *items* to send
- The target process is specified by **dest**, which is the rank of the target process in the communicator specified by **comm**
- A unique identifier of this request is stored to **req**
- This function returns immediately

Non-blocking Receive

```
MPI_Request req;  
MPI_Irecv(&buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &req);
```

```
int MPI_Irecv(void *start, int count,  
MPI_Datatype datatype, int source, int tag,  
MPI_Comm comm, MPI_Request *req)
```

- Processing continues immediately without waiting for the message to be received
- A communication request handle (`req`) is returned for handling the pending message status
- The program must call `MPI_Wait()` or `MPI_Test()` to determine when the non-blocking receive operation completes
- **Note:** it is OK to use `MPI_Isend` with the (blocking) `MPI_Recv`, and vice-versa

MPI_Test()

```
int MPI_Test(MPI_Request *request, int *flag,  
             MPI_Status *status)
```

- Checks the status of a specified non-blocking send or receive operation
- The integer `flag` parameter is set to 1 if the operation has completed, 0 if not
- For multiple non-blocking operations, there exist functions to specify any (`MPI_Testany`), all (`MPI_Testall`) or some (`MPI_Testsome`) completions
- See man pages for details

MPI_Wait()

```
int MPI_Wait(MPI_Request *request,  
             MPI_Status *status)
```

- Blocks until a specified non-blocking send or receive operation has completed
- For multiple non-blocking operations, there exists variants to specify any (`MPI_Waitany`), all (`MPI_Waitall`) or some (`MPI_Waitsome`) completions
- See man pages for details

Async send demo

```
/* mpi-async.c */
#include <stdio.h>
#include <mpi.h>
int main( int argc, char *argv[])
{
    int rank, size, buf;
    MPI_Status status;
    MPI_Request req;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );

    if (rank == 0) {
        buf = 123456;
        MPI_Isend( &buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &req);
        big_computation();
        MPI_Wait(&req, &status);
    } else if (rank == 1) {
        MPI_Recv( &buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status );
        printf("Received %d\n", buf);
    }

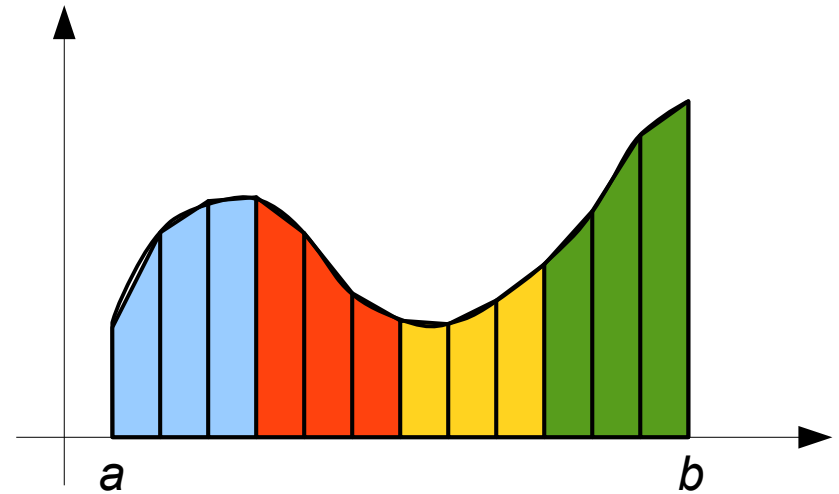
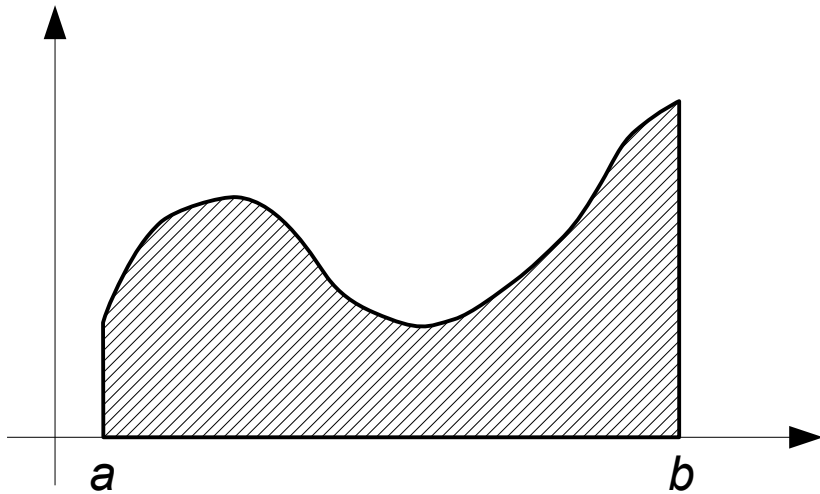
    MPI_Finalize();
    return 0;
}
```


Aborting the computation

- To abort a computation, do not use `exit()` or `abort()`: call `MPI_Abort()` instead
- `MPI_Abort(comm, err)` "gracefully" terminates all running MPI processes on communicator "comm" (e.g., `MPI_COMM_WORLD`) returning the error code "err"

Example: Trapezoid rule with MPI

The trapezoid strikes back



```
double result = 0.0;
double h = (b-a)/n;
double x = a;
for ( int i = 0; i<n; i++ ) {
    result += h*(f(x) + f(x+h))/2.0;
    x += h;
}
```

See [trap.c](#)

Parallel pseudo-code (naïve)

```
partial_result = trap(my_rank, comm_sz, a, b, n);

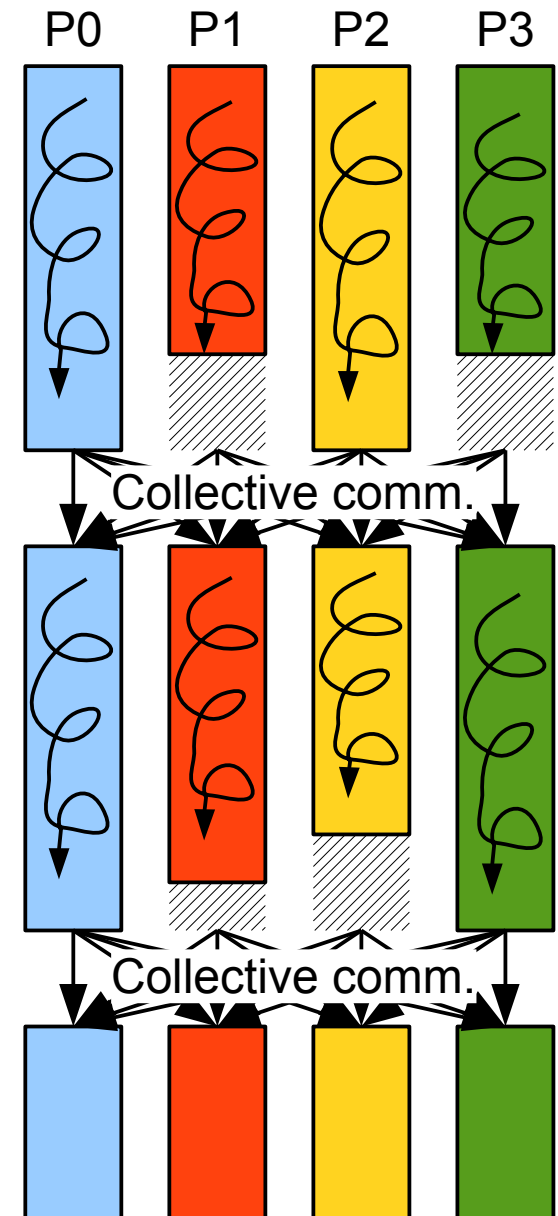
if (my_rank != 0) {
    Send partial_result to process 0;
} else {    /* my_rank == 0 */
    result = partial_result;
    for ( int p = 1; p < comm_sz; p++ ) {
        Receive partial_result from process p;
        result += partial_result;
    }
    print result;
}
```

See [mpi-trap0.c](#)

Collective Communication

Collective communications

- Send/receive operations are rarely used in practice
- Many applications use the *bulk synchronous* pattern:
 - Repeat:
 - Local computation
 - Communicate to update global view on all processes
- Collective communications are executed by all processes in the group, to compute and share some global result



Collective communications

- Collective communications are assumed to be more efficient than point-to-point operations achieving the same result
- Understanding when collective communications are to be used is an **essential skill** of a MPI programmer

MPI_Barrier()

- Executes a barrier synchronization in a group
 - When reaching the **MPI_Barrier()** call, a process blocks until all processes in the group reach the same **MPI_Barrier()** call
 - Then all processes are free to continue

MPI_Bcast()

Broadcasts a message to all other processes of a group

```
count = 3;  
src = 1; /* broadcast originates from process 1 */  
MPI_Bcast(buf, count, MPI_INT, src, MPI_COMM_WORLD);
```

Proc 0

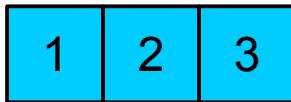
Proc 1

Proc 2

Proc 3



`buf[]` (before)



`buf[]` (after)

MPI_Scatter()

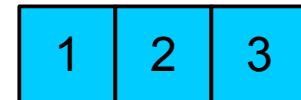
Distribute data to other processes in a group

```
sendcnt = 3; /* how many items are sent to each process */
recvcnt = 3; /* how many items are received by each process */
src = 1; /* process 1 contains the message to be scattered */
MPI_Scatter(sendbuf, sendcnt, MPI_INT,
            recvbuf, recvcnt, MPI_INT, src, MPI_COMM_WORLD);
```

sendbuf[] (before)

recvbuf[] (after)

Proc 0



Proc 1



Proc 2



Proc 3



MPI_Scatter()

- The **MPI_Scatter()** operation produces the same result as if the root executes a series of

```
MPI_Send(sendbuf + i * sendcount * extent(sendtype),  
sendcount, sendtype, i, ...)
```

and all other processes execute

```
MPI_Recv(recvbuf, recvcount, recvtype, i, ...)
```

MPI_Gather()

Gathers together data from other processes

```
sendcnt = 3; /* how many items are sent by each process */
recvcnt = 3; /* how many items are received from each process */
dst = 1;     /* message will be gathered at process 1 */
MPI_Gather(sendbuf, sendcnt, MPI_INT,
           recvbuf, recvcnt, MPI_INT, dst, MPI_COMM_WORLD);
```

sendbuf[] (before)

Proc 0	1	2	3
Proc 1	4	5	6
Proc 2	7	8	9
Proc 3	10	11	12

recvbuf[] (after)

1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

MPI_Allgather()

Gathers data from other processes and distribute to all

```
sendcnt = 3;  
recvcnt = 3;  
MPI_Allgather(sendbuf, sendcnt, MPI_INT,  
               recvbuf, recvcnt, MPI_INT, MPI_COMM_WORLD);
```

sendbuf[] (before)

Proc 0	1	2	3
Proc 1	4	5	6
Proc 2	7	8	9
Proc 3	10	11	12

recvbuf[] (after)

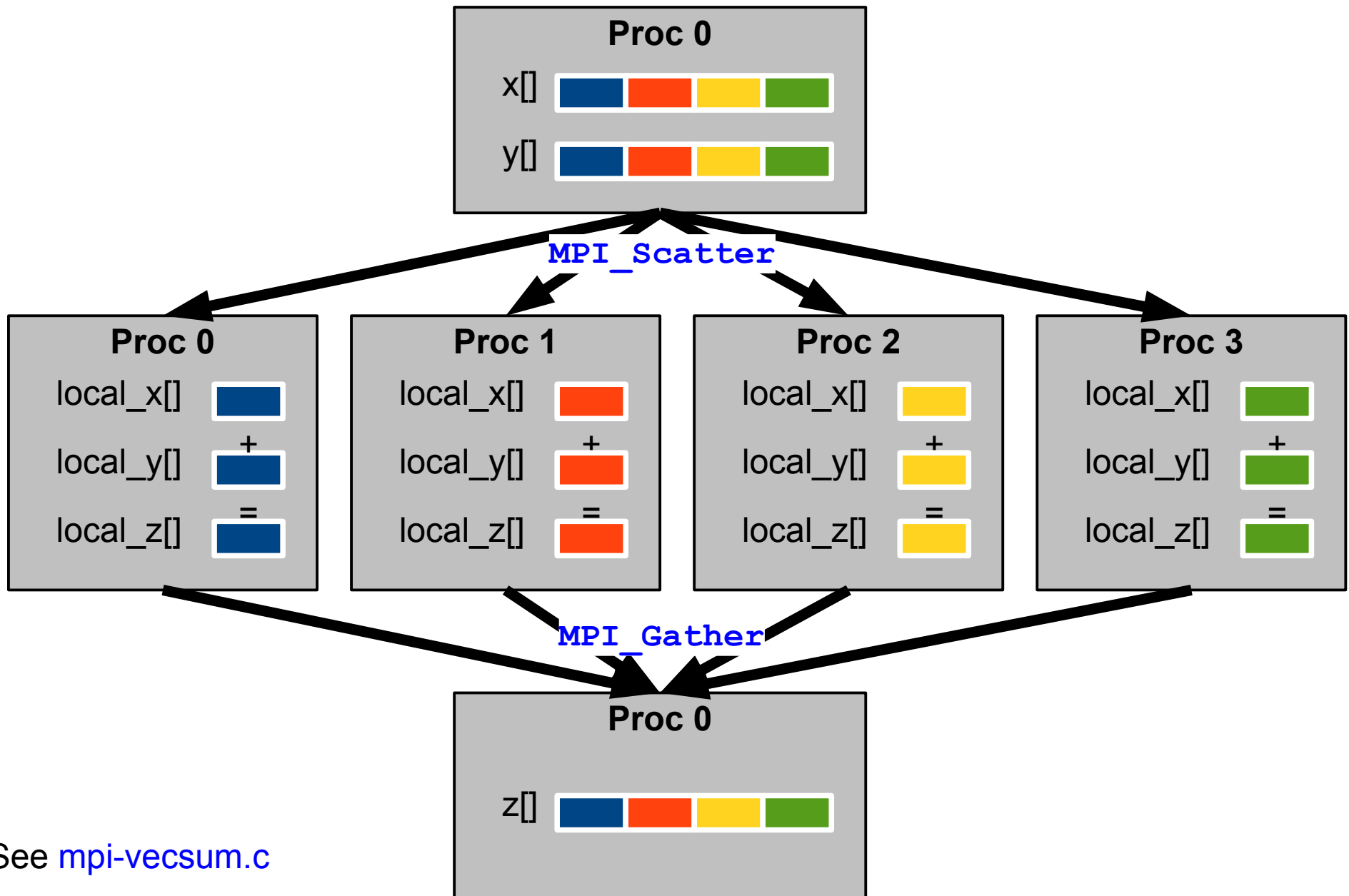
1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12

Example: Vector Sum

$$\begin{aligned} \mathbf{x} + \mathbf{y} &= (x_0, x_1, \dots, x_{n-1}) + (y_0, y_1, \dots, y_{n-1}) \\ &= (x_0 + y_0, x_1 + y_1, \dots, x_{n-1} + y_{n-1}) \\ &= (z_0, z_1, \dots, z_{n-1}) \\ &= \mathbf{z} \end{aligned}$$

```
void sum( double* x, double* y, double* z, int n )
{
    for (int i=0; i<n; i++) {
        z[i] = x[i] + y[i];
    }
}
```

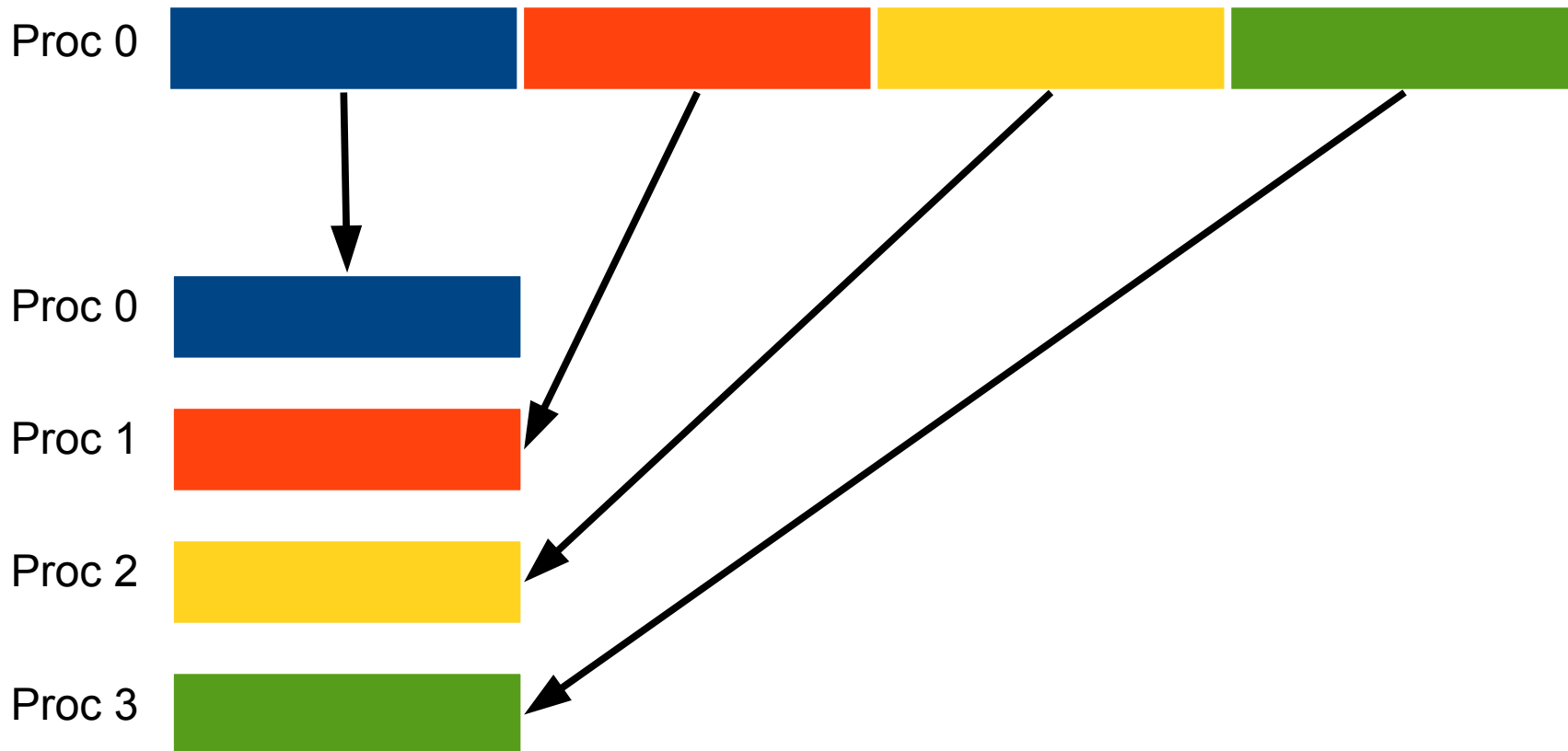
Parallel Vector Sum



See [mpi-vecsum.c](#)

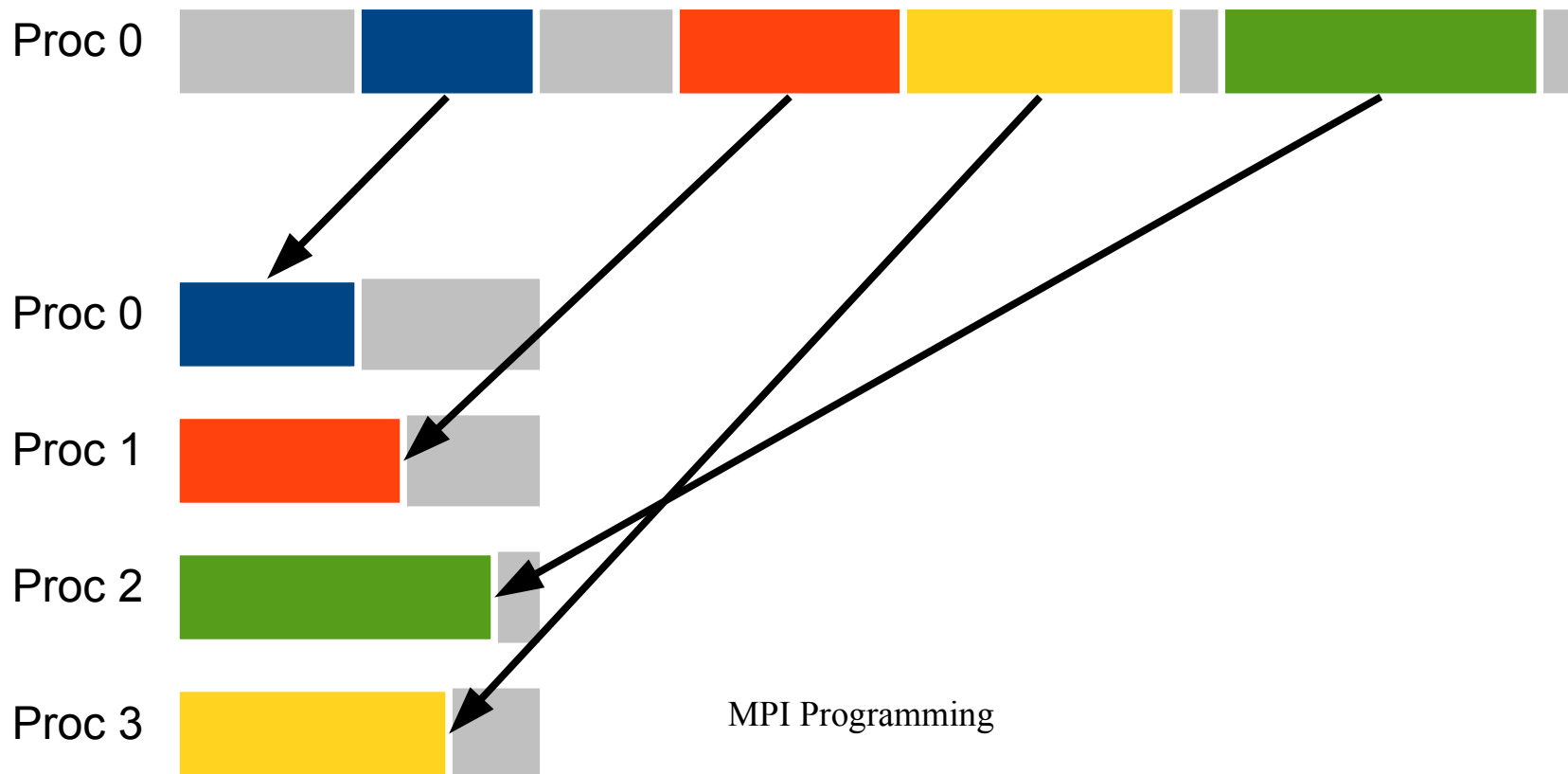
MPI_Scatter()

- Contiguous data
- Uniform message size

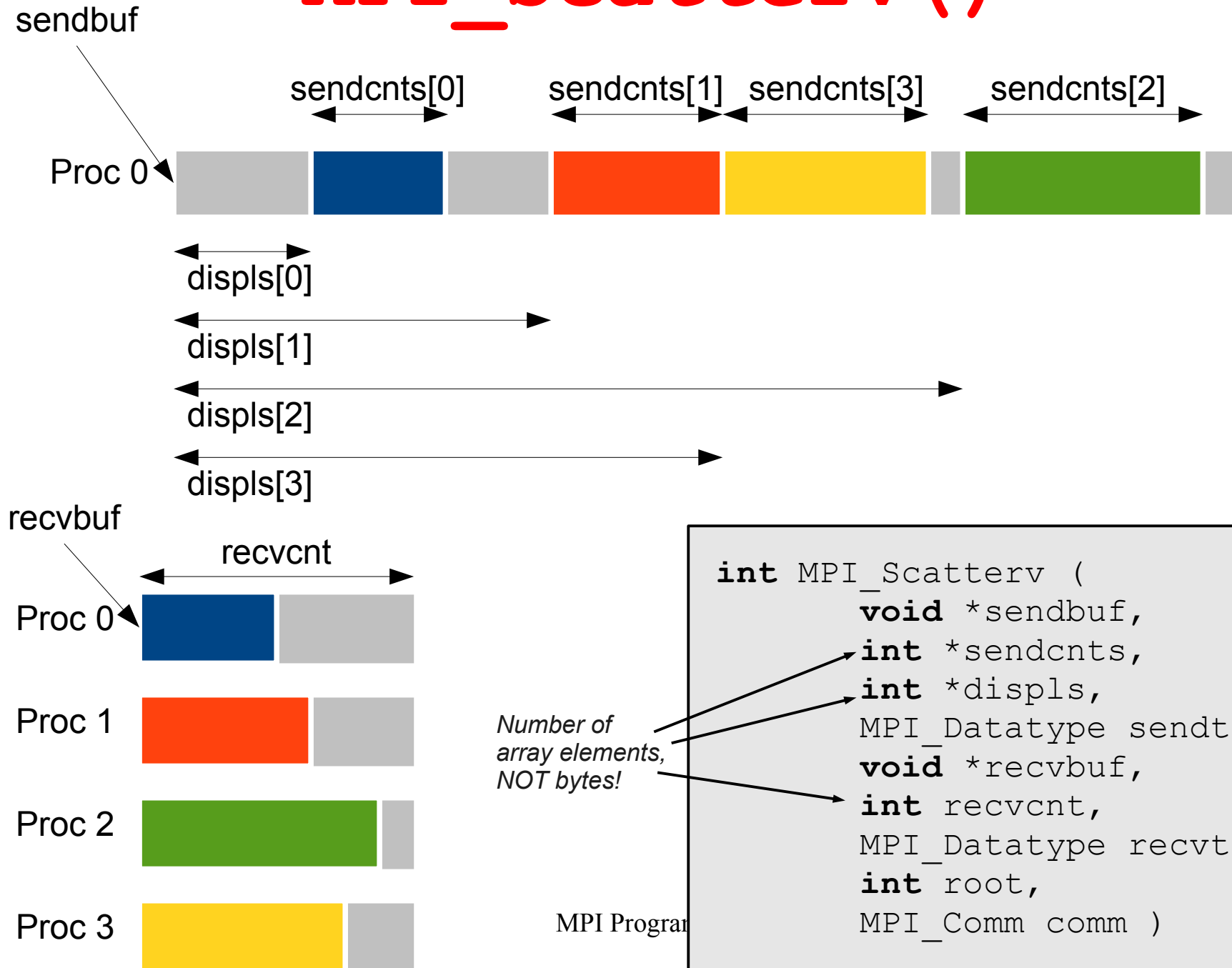


MPI_Scatterv() / MPI_Gatherv()

- Gaps are allowed between messages in source data
- Irregular message sizes are allowed
- Data can be distributed to processes in any order



MPI_Scatterv()



Number of
array elements,
NOT bytes!

MPI Program

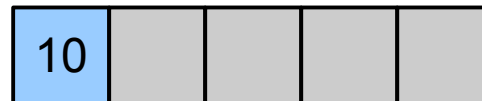
```
int MPI_Scatterv (  
    void *sendbuf,  
    int *sendcnts,  
    int *displs,  
    MPI_Datatype sendtype,  
    void *recvbuf,  
    int recvcnt,  
    MPI_Datatype recvtype,  
    int root,  
    MPI_Comm comm )
```

Example

```
int sendbuf[] = {10, 11, 12, 13, 14, 15, 16}; /* at master */
int displs[] = {3, 0, 1}; /* assume P=3 MPI processes */
int sendcnts[] = {3, 1, 4};
int recvbuf[5];
...
MPI_Scatterv(sendbuf, sendcnts, displs, MPI_INT, recvbuf, 5,
MPI_INT, 0, MPI_COMM_WORLD);
```



Proc 0



Proc 1



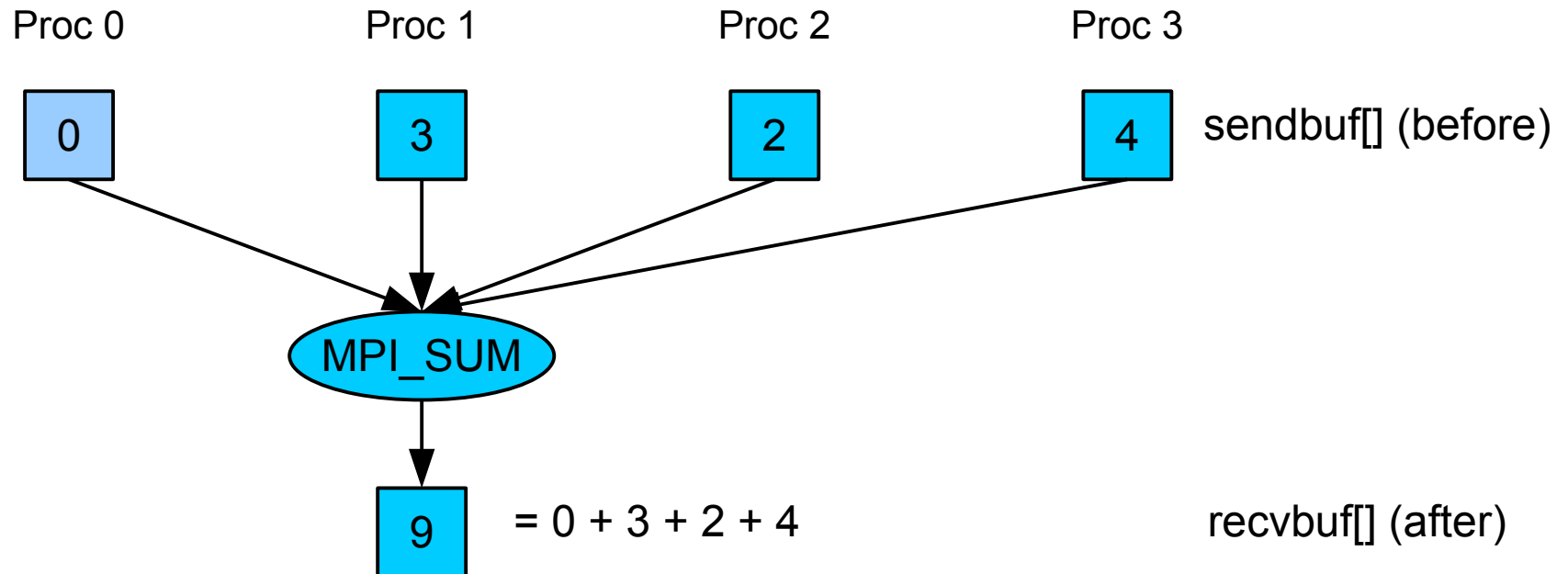
Proc 2

recvbuf[]

MPI_Reduce()

Performs a reduction and place result in one process

```
count = 1;  
dst = 1; /* result will be placed in process 1 */  
MPI_Reduce(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, dst, MPI_COMM_WORLD);
```



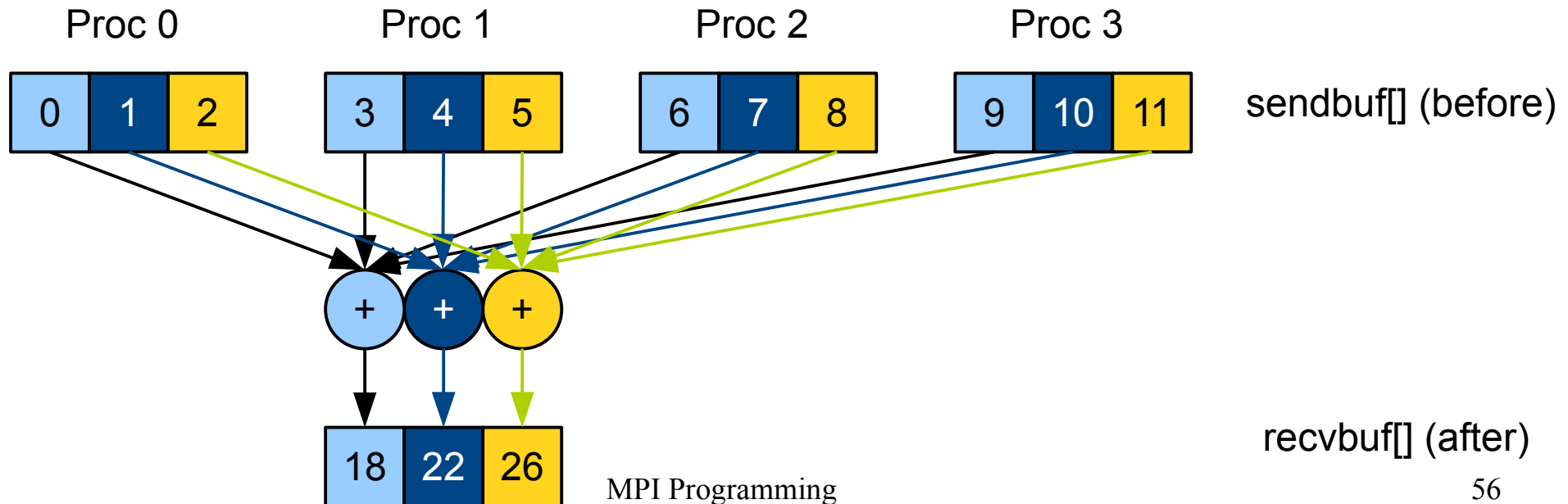
Predefined reduction operators

Operation Value	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical AND
MPI_BAND	Bitwise AND
MPI_LOR	Logical OR
MPI_BOR	Bitwise OR
MPI_LXOR	Logical exclusive OR
MPI_BXOR	Bitwise exclusive OR
MPI_MAXLOC	Maximum and location of maximum
MPI_MINLOC	Minimum and location of minimum

MPI_Reduce()

- If $\text{count} > 1$, `recvbuf[i]` is the reduction of all elements `sendbuf[i]` at the various processes

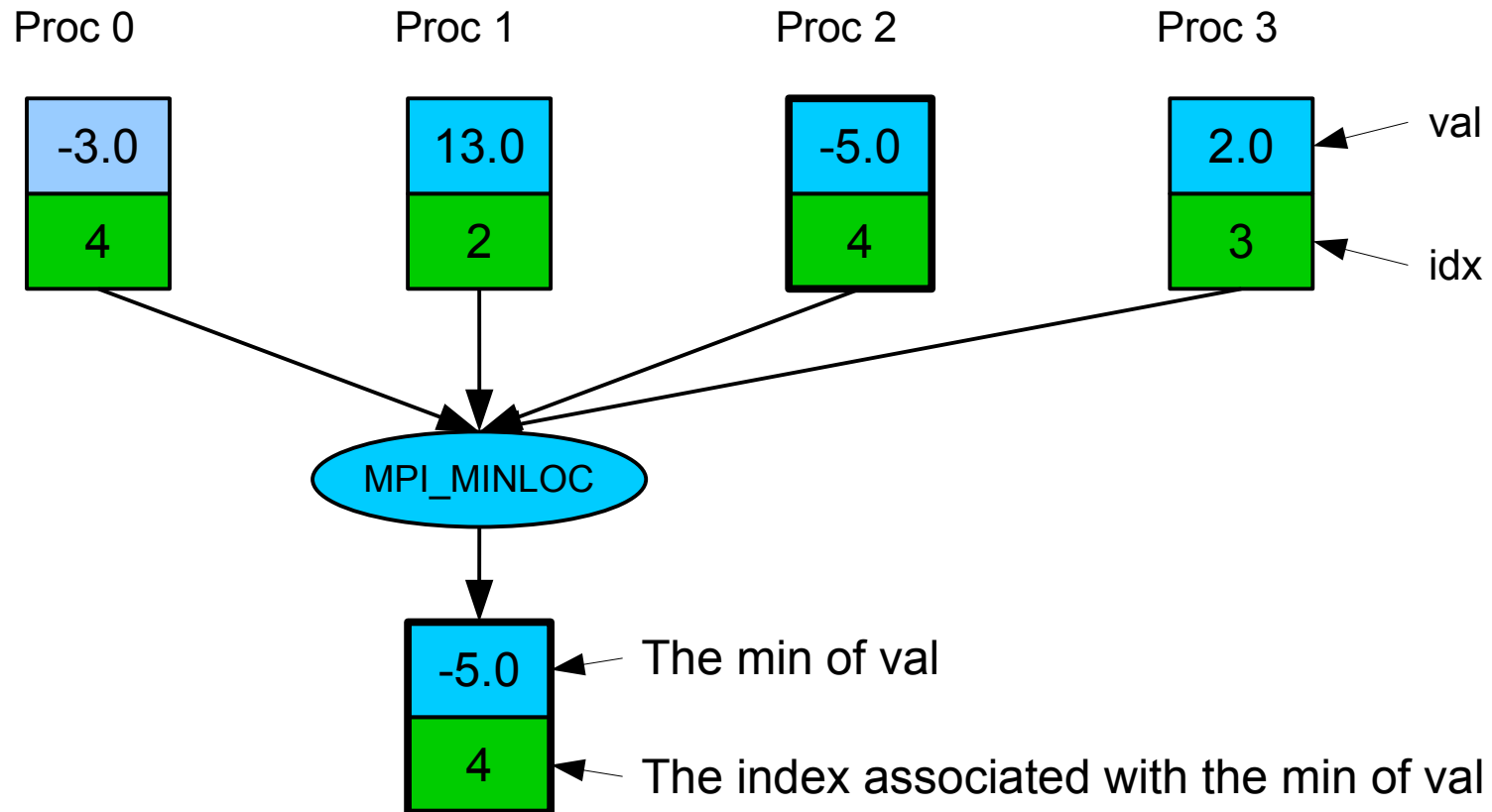
```
count = 3;  
dst = 1;  
MPI_Reduce(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, dst, MPI_COMM_WORLD);
```



MINLOC and MAXLOC

Compute global min/max and return an associated index

```
struct {double val; int idx} in, out;  
dst = 1; /* result will be placed in process 1 */  
MPI_Reduce(&in, &out, 1, MPI_DOUBLE_INT, MPI_MINLOC, dst, MPI_COMM_WORLD);
```



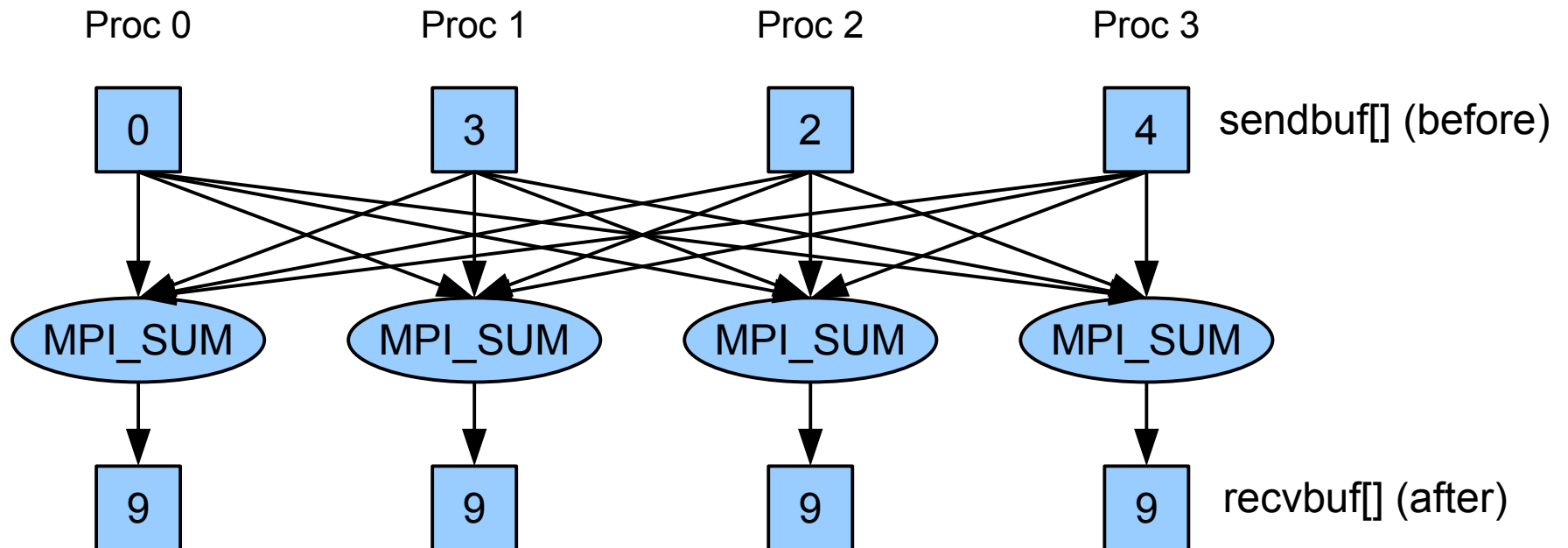
Parallel trapezoid (with reduction)

- See [mpi-trap1.c](#)

MPI_Allreduce()

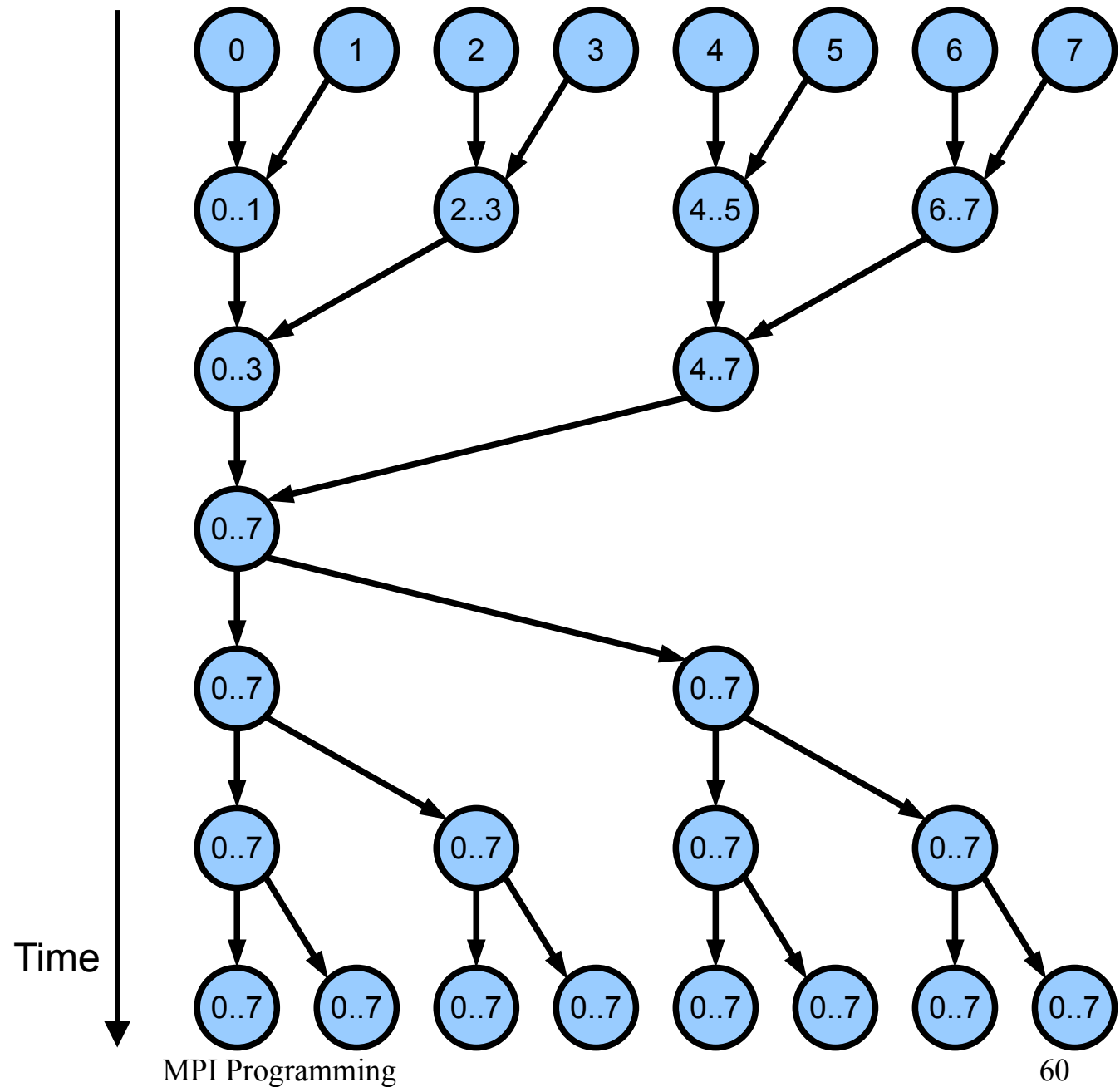
Performs a reduction and place result in all processes

```
count = 1;  
MPI_Allreduce(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```



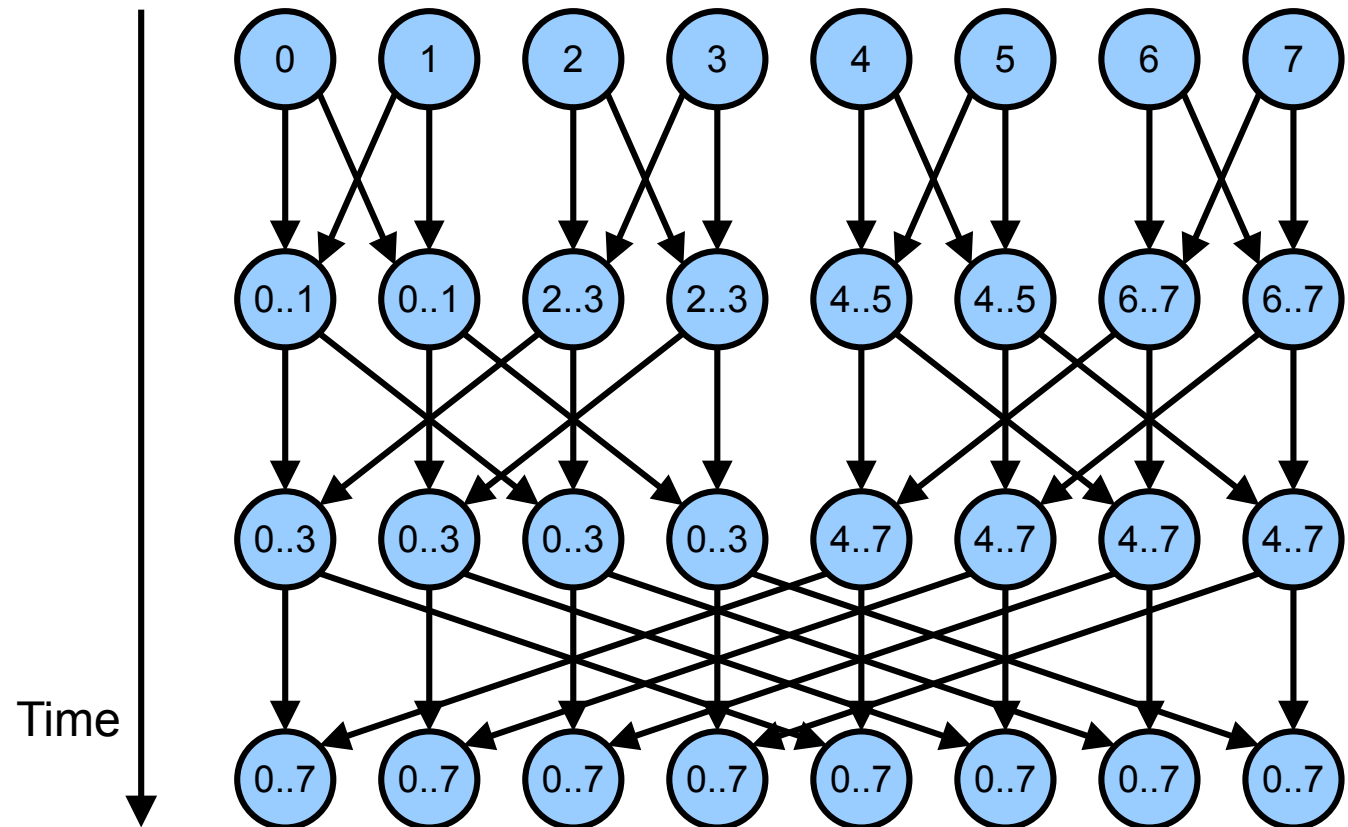
MPI_Allreduce()

One possible
implementation
using tree-
structured
communication.



MPI_Allreduce()

Another
implementation
with better span,
using a *butterfly*
communication
pattern.



MPI_Alltoall()

Each process performs a scatter operation

```
sendcnt = 2;  
recvcnt = 2;  
MPI_Alltoall(sendbuf, sendcnt, MPI_INT,  
             recvbuf, recvcnt, MPI_INT, MPI_COMM_WORLD);
```

sendbuf[] (before)

Proc 0	1	2	3	4	5	6	7	8
Proc 1	9	10	11	12	13	14	15	16
Proc 2	17	18	19	20	21	22	23	24
Proc 3	25	26	27	28	29	30	31	32

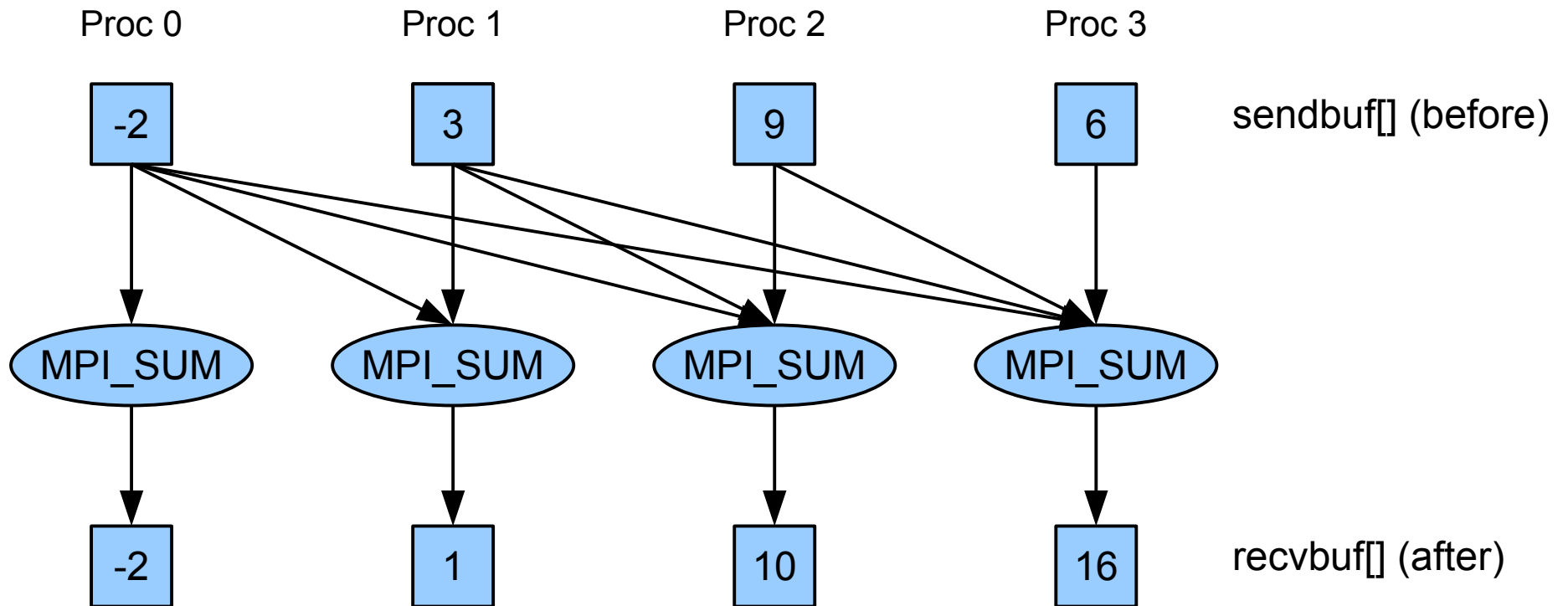
recvbuf[] (after)

1	2	9	10	17	18	25	26
3	4	11	12	19	20	27	28
5	6	13	14	21	22	29	30
7	8	15	16	23	24	31	32

MPI_Scan()

Inclusive scan

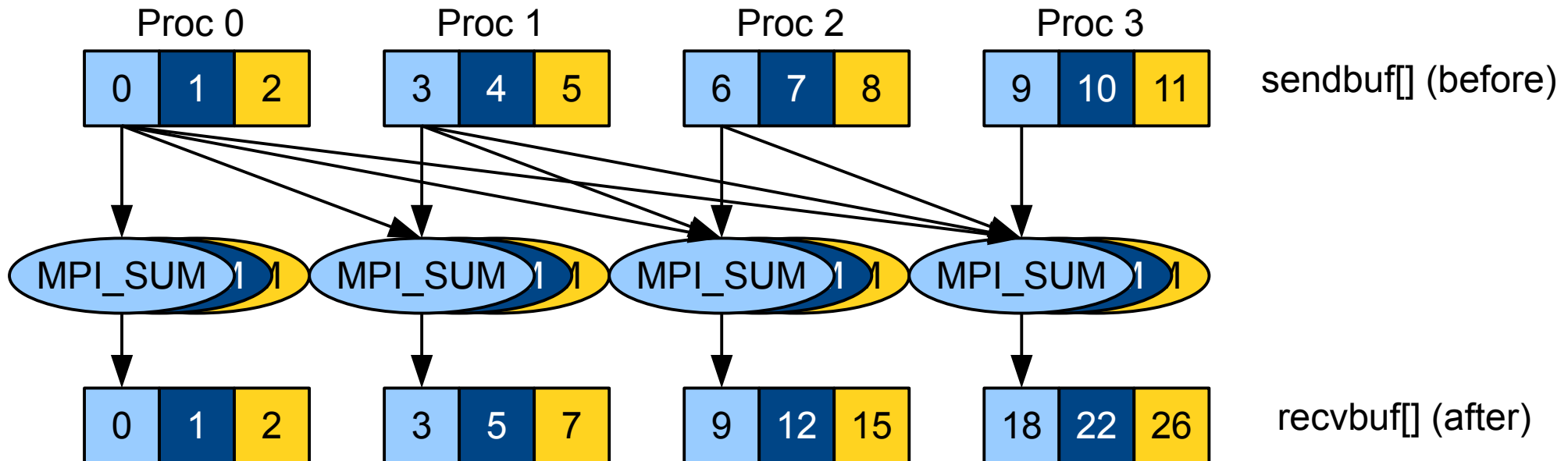
```
count = 1;  
MPI_Scan(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```



MPI_Scan()

- If $\text{count} > 1$, `recvbuf[i]` at proc. j is the scan of all elements `sendbuf[i]` at the first j processes (incl.)

```
count = 3;  
MPI_Scan(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```



Collective Communication Routines / 1

MPI_Barrier(comm)

- Synchronization operation. Creates a barrier synchronization in a group. Each process, when reaching the MPI_Barrier call, blocks until all processes in the group reach the same MPI_Barrier call. Then all processes can continue.

MPI_Bcast(buffer, count, datatype, root, comm)

- Broadcasts (sends) a message from the process with rank "root" to all other processes in the group.

MPI_Scatter(sendbuf, sendcnt, sendtype, recvbuf, recvcnt, recvtype, root, comm)

MPI_Scatterv(sendbuf, sendcnts[], displs[], sendtype, recvbuf, int recvcnt, recvtype, root, comm)

- Distributes distinct messages from a single source process to each process in the group

MPI_Gather(sendbuf, sendcnt, sendtype, recvbuf, recvcount, recvtype, root, comm)

MPI_Gatherv(sendbuf, sendcnt, sendtype, recvbuf, recvcnts[], displs[], recvtype, root, comm)

- Gathers distinct messages from each process in the group to a single destination process. This routine is the reverse operation of MPI_Scatter

MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)

- Concatenation of data to all processes in a group. Each process in the group, in effect, performs a one-to-all broadcasting operation within the group

MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm)

- Applies a reduction operation on all processes in the group and places the result in one process

Collective Communication Routines / 2

`MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm)`

- Collective computation operation + data movement. Does an element-wise reduction on a vector across all processes in the group; all processes receive the result. This is logically equivalent to a `MPI_Reduce()` followed by `MPI_Bcast()`

`MPI_Scan(sendbuf, recvbuf, count, datatype, op, comm)`

- Performs a scan with respect to a reduction operation across a process group.

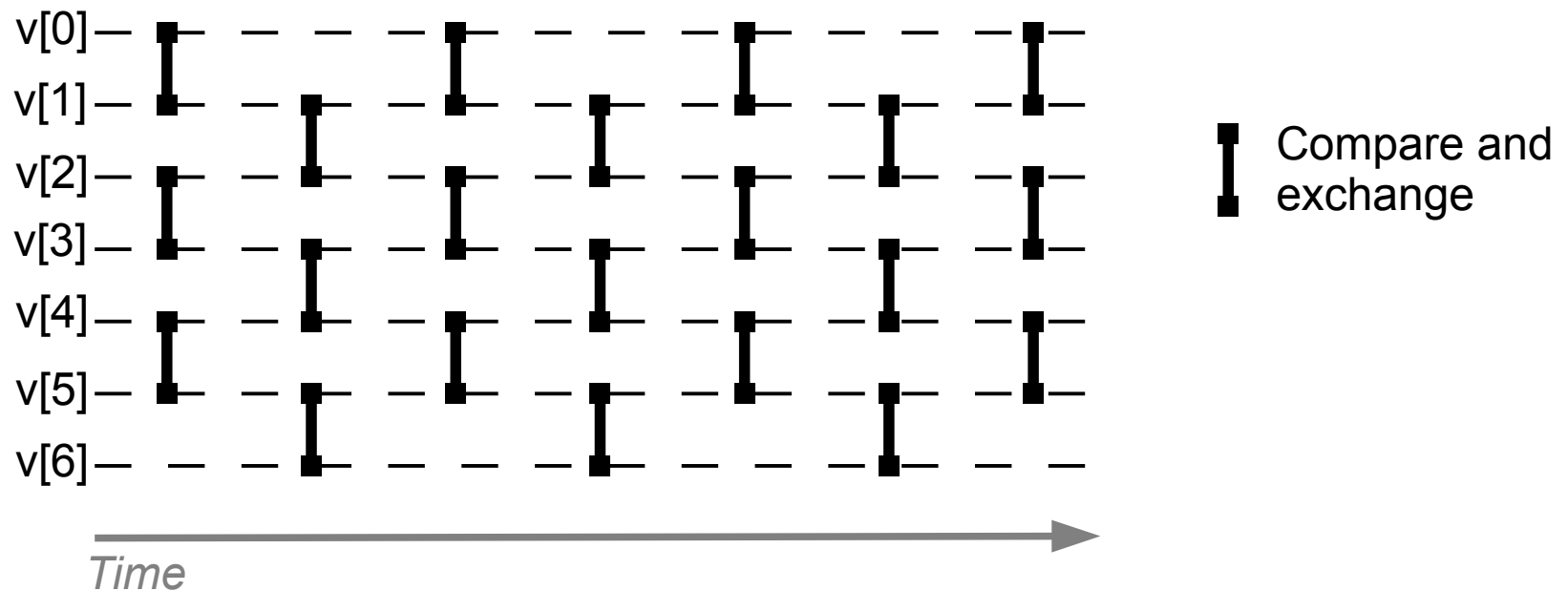
`MPI_Alltoall(sendbuf, sendcount, sendtype, recvbuf, recvcnt, recvtype, comm)`

- Data movement operation. Each process in a group performs a scatter operation, sending a distinct message to all the processes in the group in order by index.

Odd-Even sort with MPI

Odd-Even Transposition Sort

- Variant of bubble sort
- First, compare all (even, odd) pairs of adjacent elements; exchange them if in the wrong order
- Then compare all (odd, even) pairs, exchanging if necessary; repeat the step above



Choosing the Granularity

- Communication in distributed memory systems is very expensive, therefore working on individual array elements is not appropriate
- We achieve a coarser granularity by splitting the array into **blocks** and working at the block level

G. Baudet and D. Stevenson, "*Optimal Sorting Algorithms for Parallel Computers*," in IEEE Transactions on Computers, vol. C-27, no. 1, pp. 84-87, Jan. 1978. doi:10.1109/TC.1978.1674957

Odd-Even Transposition Sort

- The array is split in blocks that are assigned to MPI processes
- Each process sorts its block (e.g., using `qsort`)
- At each exchange-swap step:
 - Process i sends a copy of its chunk to process $i + 1$
 - Process $i + 1$ sends a copy of its chunk to process i
 - Process i merges the chunks, discards **upper** half
 - Process $i + 1$ merges the chunks, discards **lower** half

Odd-Even Transposition Sort

Process i

Process $i + 1$

10 7 12 33 9

13 52 14 97 31

sort

sort

7 9 10 12 33

13 14 31 52 97

Send to partner

7 9 10 12 33 13 14 31 52 97

7 9 10 12 33 13 14 31 52 97

merge

merge

7 9 10 12 13 14 31 33 52 97

7 9 10 12 13 14 31 33 52 97

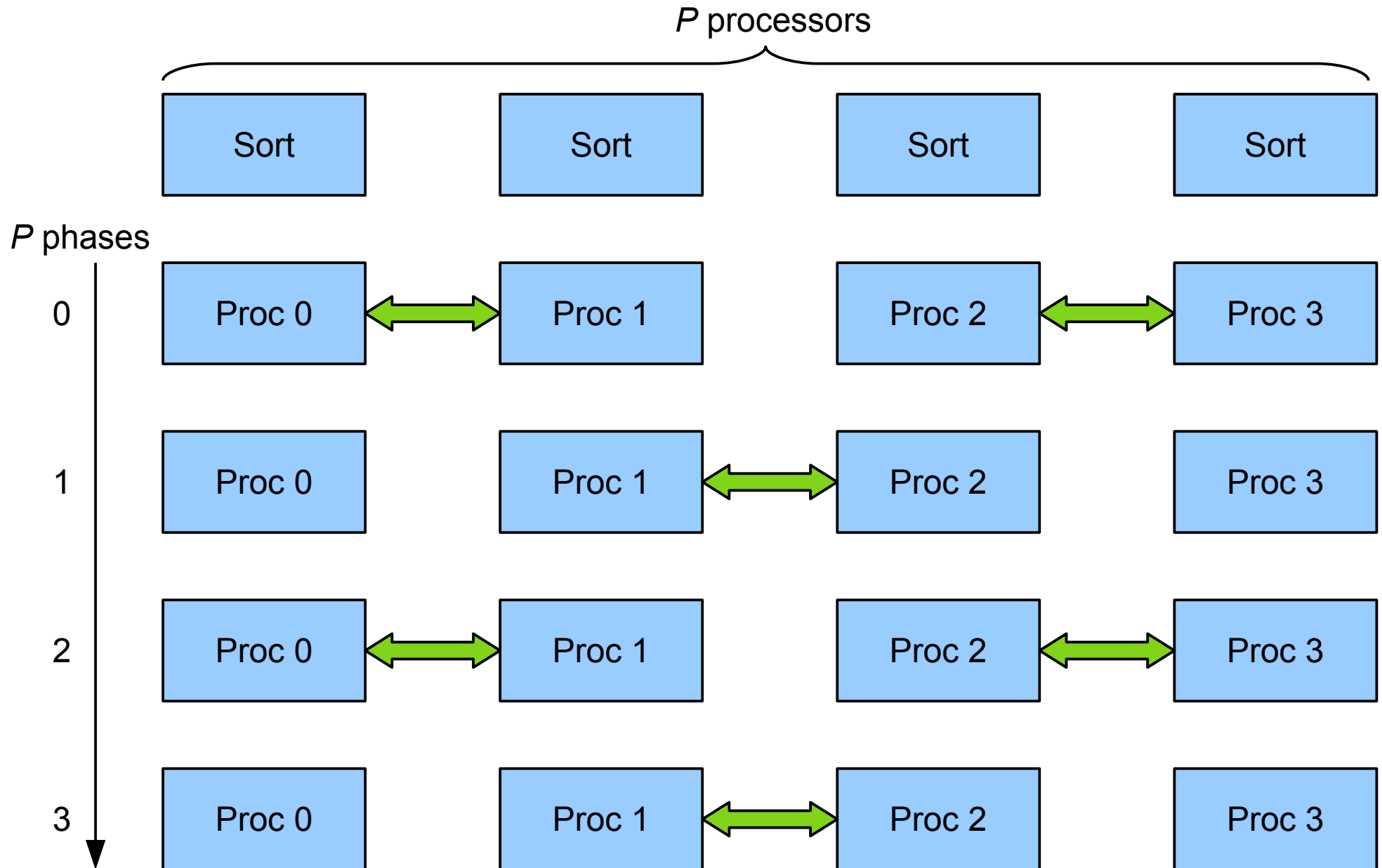
discard upper half

discard lower half

7 9 10 12 13

14 31 33 52 97

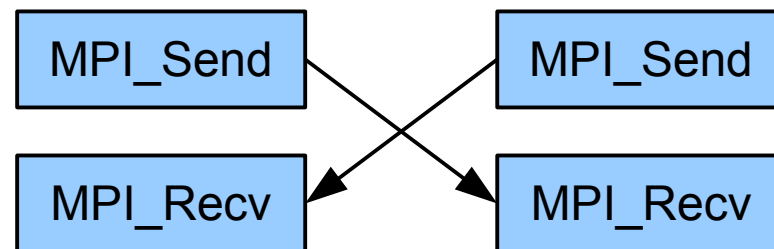
Communication pattern for odd-even sort



Beware of the (possible) deadlock!!

```
Sort local values
for (phase=0; phase < comm_sz; phase++) {
    partner = compute_partner(phase, my_rank);
    if (I am not idle) {
        → Send my keys to partner;
        Receive keys from partner;
        if (my_rank < partner) {
            Keep smaller keys;
        } else {
            Keep larger keys;
        }
    }
}
```

Depending on the MPI implementation, the send operation may block if there is no matching receive at the other end; unfortunately, all receive are executed only after the send completes!



Solution 1 (ugly): restructure communications

```
MPI_Send(msg1, size, MPI_INT, partner, 0, comm);  
MPI_Recv(msg2, size, MPI_INT, partner, 0, comm, MPI_STATUS_IGNORE);
```



```
if ( my_rank % 2 == 0 ) {  
    MPI_Send(msg1, size, MPI_INT, partner, 0, comm);  
    MPI_Recv(msg2, size, MPI_INT, partner,  
              0, comm, MPI_STATUS_IGNORE);  
} else {  
    MPI_Recv(msg2, size, MPI_INT, partner,  
              0, comm, MPI_STATUS_IGNORE);  
    MPI_Send(msg1, size, MPI_INT, partner, 0, comm);  
}
```


Solution 2 (better): use async Send

```
MPI_Send(msg1, size, MPI_INT, partner, 0, comm);  
MPI_Recv(msg2, size, MPI_INT, partner, 0, comm, MPI_STATUS_IGNORE);
```



```
MPI_Request req;  
MPI_Isend(msg1, size, MPI_INT, partner, 0, comm, &req);  
MPI_Recv(msg2, size, MPI_INT, partner, 0, comm, MPI_STATUS_IGNORE)  
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

Solution 3 (best): **`MPI_Sendrecv()`**

- Executes a blocking send and a receive in a single call
 - *dest* and the *source* can be the same or different
 - MPI schedules the communications so that the program won't hang or crash
- **`MPI_Sendrecv()`** can be matched by **`MPI_Send()`** / **`MPI_Recv()`**
 - However, it is **very unlikely** that you will ever need to do that

```
int MPI_Sendrecv(  
    void*      sendbuf,  
    int        sendcount,  
    MPI_Datatype sendtype,  
    int        dest,  
    int        sendtag,  
    void*      recvbuf,  
    int        recvcount,  
    MPI_Datatype recvtype,  
    int        source,  
    int        recvtag,  
    MPI_Comm   comm,  
    MPI_Status* status  
)
```

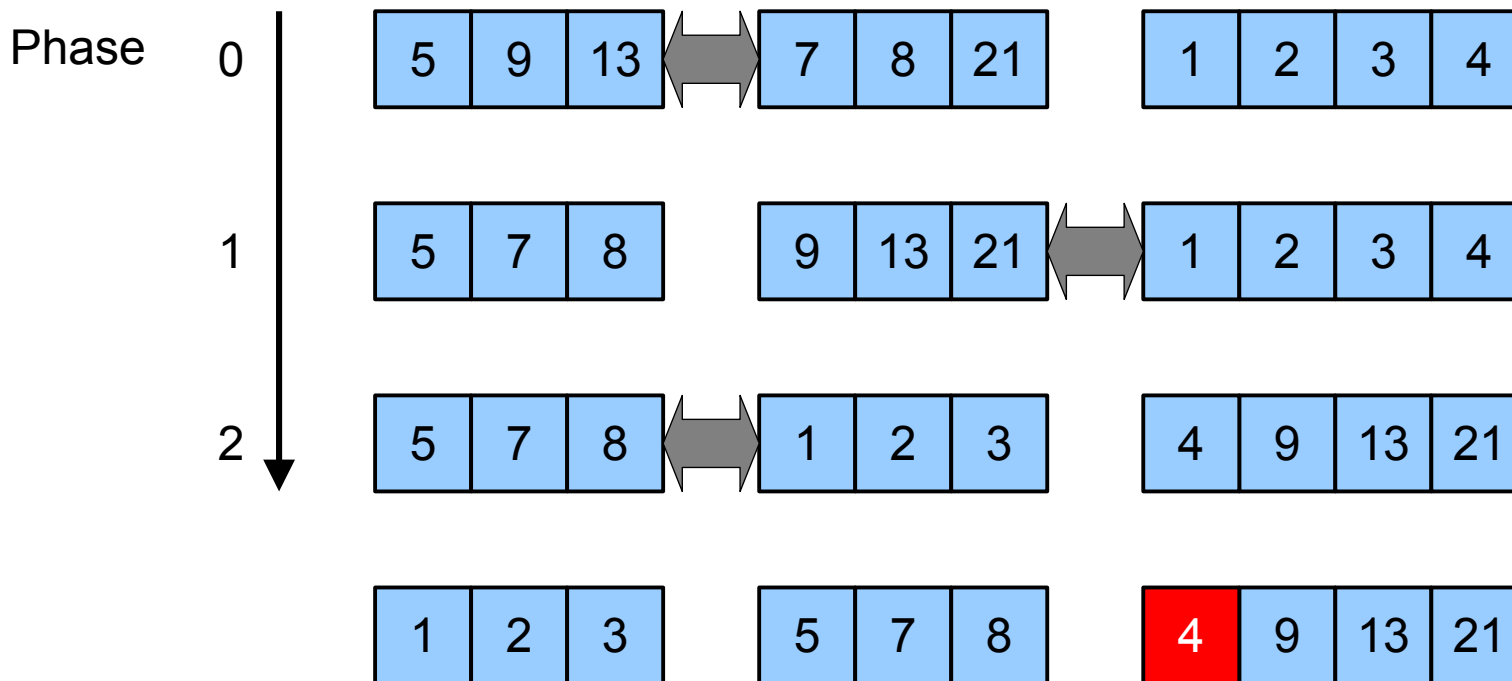
See [mpi-odd-even.c](#)

Odd-even sort with arbitrary array length

- If the array length is not a multiple of `comm_sz`, then care must be taken.
- When processes exchange chunks, they must also **exchange their sizes**.
- Example:
 - Process i has a chunk C_i of length L_i
 - Process $i + 1$ has a chunk C_{i+1} of length L_{i+1}
 - Process i :
 - Keeps the smallest L_{i+1} elements of $C_i \cup C_{i+1}$
 - Process $i + 1$:
 - Keeps the largest L_i elements of $C_i \cup C_{i+1}$

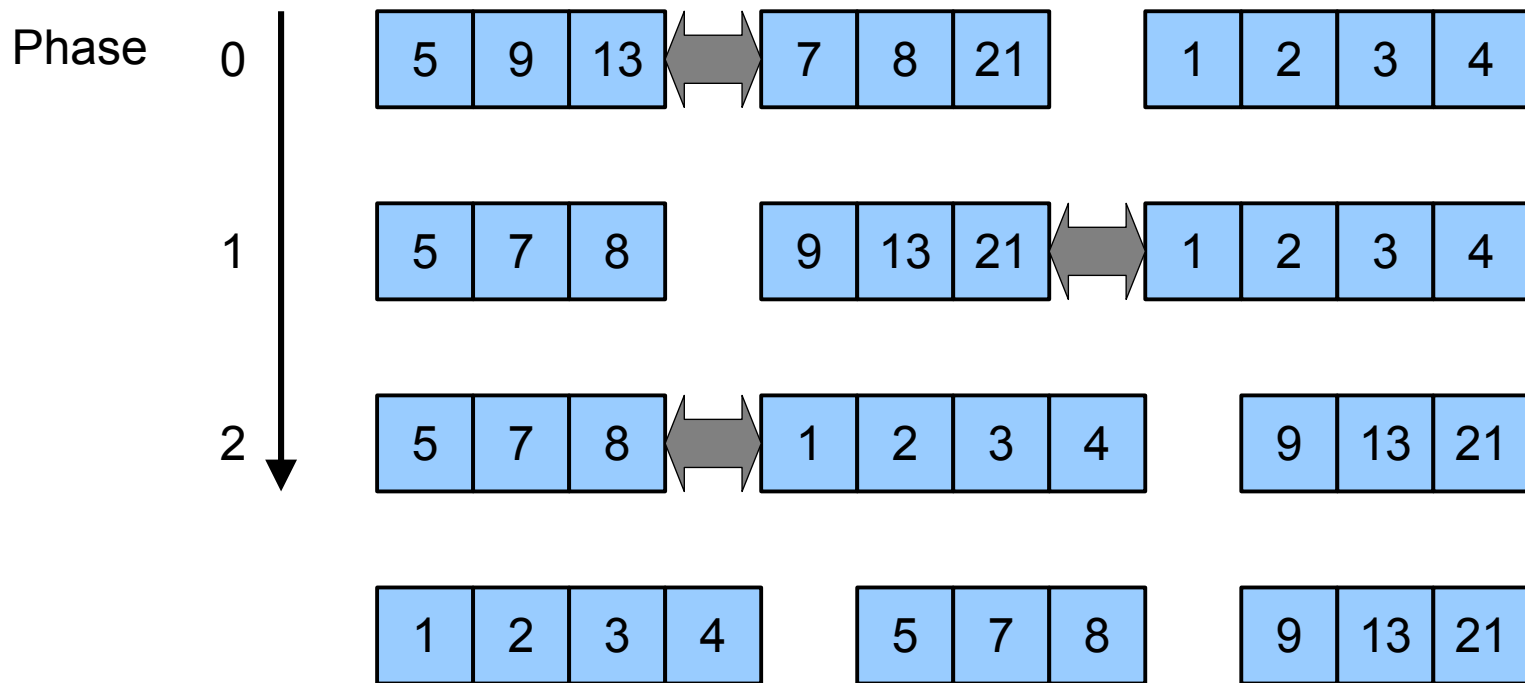
Counterexample

- If we do not exchange the sizes, the array is not sorted properly at the end of the exchange-merge phases.



Counterexample

- If we do exchange the sizes, the result is correct.



Concluding Remarks

- Message passing is a very basic model
 - Extremely low level; heavy weight
 - Expense comes from communication and lots of local code
 - Communication code is often more than half
 - Tough to make adaptable and flexible
 - Tough to get right
- Programming model of choice for scalability
 - Widespread adoption due to portability